

EXERCISE 1 — ELECTRONIC DEVICE MANAGEMENT

1. Overview

Build a Java Console application to manage electronic devices sold in a store. Data is managed with an in-memory list (`ArrayList`). The program menu must run continuously until the user chooses to exit.

2. Class Structure

2.1. Abstract Class `ElectronicDevice` (parent class)

Manages the common information of every electronic device:

- Device ID
- Device Name
- Manufacturer
- Selling Price
- Import Date (`LocalDate importDate`, entered in yyyy-MM-dd format)

`ElectronicDevice` is an **abstract class**, declaring the abstract method `displayInfo()` which every child class must override (this must also display the import date).

Inside `ElectronicDevice` there is a **static nested class** `DeviceStatistics` responsible for statistics:

- Count of total devices
- Count of Phones
- Count of Laptops
- Average selling price of all devices

2.2. Class `Phone` (extends `ElectronicDevice`)

Additional attributes:

- Battery Capacity (must be greater than 0)
- 5G Support (yes/no)

Overrides `displayInfo()`. Implements the `Discountable` interface with a **10%** discount.

2.3. Class `Laptop` (extends `ElectronicDevice`)

Additional attributes:

- RAM (must be greater than 0)
- Screen Size

Overrides `displayInfo()`. Implements the `Discountable` interface with a **15%** discount.

2.4. Interface Discountable

```
double calculateDiscountPrice();
```

3. Custom Exceptions

Exception	When it is thrown
DuplicateDeviceIdException	The user enters a device ID that already exists when adding a new device
InvalidPriceException	The entered selling price is not greater than 0
Exception for invalid type-specific attribute	Battery capacity (Phone) or RAM (Laptop) is not greater than 0
DeviceNotFoundException	No device is found matching the requested ID

The program must use try-catch to handle incorrect input data types; it must not terminate abnormally when an error occurs, and must instead allow the user to re-enter the value.

4. Program Functions (consolidated menu)

CRUD

1. Add a device (Phone or Laptop) — check that the ID is not a duplicate (DuplicateDeviceIdException), price > 0 (InvalidPriceException), the type-specific attribute is valid, and enter the import date in yyyy-MM-dd format.
2. Search for a device by ID — returns Optional<ElectronicDevice> (using the Stream API); if found, display the information, otherwise display a “not found” message (no exception thrown on this normal search path).

Display / Filtering (using Lambda + Stream API)

3. Display the list of phones that support 5G.
4. Display the list of laptops with RAM of 16GB or more.
5. Find the phone with the largest battery capacity.
6. Find the laptop with the largest screen size.
7. Find the device with the highest selling price (using Stream + Optional; if the list is empty, show an appropriate message instead of an error).
8. Compute the total value of all devices.
9. Find devices by manufacturer (the user enters the manufacturer’s name).
10. Display the list of device names (using map()).
11. Display the list of devices imported in a year specified by the user.
12. Display the discounted price of all devices (ID, Name, Original Price, Discounted Price).

Sorting

13. Sort devices by name (A – Z) — implemented with a **Lambda Expression combined with a Comparator** when calling Collections.sort()/List.sort(). (No standalone Comparator class is created — this requirement supersedes the earlier approach of using a separate Comparator class.)

- Sort devices by selling price in ascending order — implemented the same way, with a Lambda Expression.

Statistics

- Display device statistics (via `DeviceStatistics`): total number of devices, number of Phones, number of Laptops, average selling price.

5. Consolidated Technical Requirements

- Apply inheritance to reuse common attributes; clearly separate the shared functionality from the type-specific functionality of each device type.
- `ElectronicDevice` is an abstract class; `Phone/Laptop` override `displayInfo()`.
- Apply the `Discountable` interface to `Phone` and `Laptop`.
- Apply Lambda Expressions and the Stream API to all filtering/searching/sorting/computation operations on the device list (replacing traditional for-loops and standalone Comparator classes).
- Use Method References instead of Lambda Expressions wherever appropriate.
- Use `Optional<ElectronicDevice>` for search functions.
- Use custom exceptions (`DuplicateDeviceIdException`, `InvalidPriceException`, `DeviceNotFoundException`, and the exception for invalid type-specific attributes) together with `try-catch` so the program never terminates abnormally.
- The `importDate` attribute is of type `LocalDate`, entered/displayed in `yyyy-MM-dd` format.

EXERCISE 2 — TRANSPORTATION MANAGEMENT SYSTEM

1. Overview

Build a Java Console application to manage transportation vehicles registered in a transportation center. All objects (both `Car` and `Motorcycle`) are stored in a single `ArrayList<Vehicle>`, and the program must run continuously until the user chooses to exit.

2. Class Structure

2.1. Abstract Class `Vehicle` (parent class)

Common information for every vehicle:

- Vehicle ID
- Vehicle Name
- Manufacturer
- Vehicle Value
- Registration Date (`LocalDate`, entered as `yyyy-MM-dd`, validated — invalid/incorrectly formatted input must be caught and re-prompted, e.g. *“Invalid date format. Please enter the date using yyyy-MM-dd.”*)

Vehicle is an **abstract class** implementing:

- Comparable<Vehicle> — natural ordering by Vehicle Value (ascending)
- public abstract double calculateAnnualTax(); — overridden by each child class

Each child class must override a display method (e.g. displayInfo()) to show object-specific information, invoked polymorphically through a Vehicle reference wherever possible.

Vehicle also contains:

- A **static nested class** VehicleStatistics, automatically updated whenever a vehicle is added, exposing:
 - public static int getTotalVehicles();
 - public static int getTotalCars();
 - public static int getTotalMotorcycles();
 - public static double getTotalVehicleValue();
- A **non-static inner class** VehicleInsurance with attributes Insurance Provider and Coverage Amount. Every Vehicle object must contain exactly one VehicleInsurance object, entered by the user at creation time and shown as part of displayInfo().

2.2. Class Car (extends Vehicle)

Additional attributes:

- Number of Seats (must be > 0)
- Fuel Type (enum: Gasoline / Diesel / Electric)

Annual tax (calculateAnnualTax()), based on fuel type:

Fuel Type	Tax Rate
Gasoline	5% of vehicle value
Diesel	6% of vehicle value
Electric	3% of vehicle value

Registration status (getRegistrationStatus(), from interface Registrable):

Condition	Status
Electric	Green Vehicle
Gasoline / Diesel	Standard Vehicle

2.3. Class Motorcycle (extends Vehicle)

Additional attributes:

- Engine Capacity in cc (must be > 0)
- ABS Supported (Yes/No)

Annual tax (`calculateAnnualTax()`), based on engine capacity:

Engine Capacity	Tax Rate
Less than 150cc	2% of vehicle value
150cc or greater	4% of vehicle value

Registration status (`getRegistrationStatus()`, from interface `Registrable`):

Condition	Status
ABS Supported	Safety Certified
No ABS	Basic Certified

2.4. Interface `Registrable`

```
public interface Registrable {  
    String getRegistrationStatus();  
}
```

Implemented by both `Car` and `Motorcycle`.

3. Custom Exceptions

Exception	Trigger
<code>DuplicateVehicleIdException</code>	User attempts to add a vehicle whose Vehicle ID already exists
<code>InvalidVehicleValueException</code>	Vehicle Value ≤ 0 , Number of Seats ≤ 0 , or Engine Capacity ≤ 0

All exceptions must be handled gracefully (appropriate error message, allow re-entry of the value); the program must keep running when an exception occurs, and must not crash on invalid-type user input.

4. Program Functions (consolidated menu)

CRUD / core operations

1. Add a Car — validated against `DuplicateVehicleIdException` and `InvalidVehicleValueException`; also collects Registration Date and Vehicle Insurance info.
2. Add a Motorcycle — same validation and data collection as above.
3. Display All Vehicles.
4. Search Vehicle by ID — refactored to use Stream API and return `Optional<Vehicle>` (no more `null`); if found show full information, otherwise display “*Vehicle not found.*”
5. Search Vehicle by Name — same Stream API / `Optional<Vehicle>` approach.

Filtering (Stream API)

6. Display All Electric Cars.
7. Display All Motorcycles with ABS.

8. Display Vehicles by Manufacturer.

Sorting

9. Sort Vehicles by Value (ascending) — via natural ordering (`Comparable<Vehicle>`).
10. Sort by Vehicle Name (A → Z).
11. Sort by Manufacturer (A → Z).
12. Sort by Annual Tax (Highest → Lowest).

(Sorting is implemented directly with Lambda Expressions passed to `Collections.sort()`/`List.sort()`. The originally-specified standalone Comparator classes — `VehicleNameComparator`, `ManufacturerComparator`, `AnnualTaxComparator` — are superseded by this Lambda-based approach and must no longer be used.)

Reporting

13. Display Vehicle Tax Report — for each vehicle: Vehicle ID, Vehicle Name, Vehicle Type, Annual Tax, Registration Status (via polymorphic calls to `calculateAnnualTax()` and `getRegistrationStatus()`).
14. Display Vehicle Statistics — via `VehicleStatistics`: Total Vehicles, Total Cars, Total Motorcycles, Total Vehicle Value.
15. Display Vehicles Registered Within the Last N Days — user enters N; show vehicles whose registration date falls within that window from today.

5. Consolidated Technical Requirements

- The program must run continuously until the user chooses to exit; Vehicle IDs must be unique.
- Use inheritance to reuse common attributes/behavior; use polymorphism by storing both Car and Motorcycle in a single `ArrayList<Vehicle>`.
- Vehicle is abstract (`calculateAnnualTax()` abstract method) and implements `Comparable<Vehicle>`; Car/Motorcycle implement `Registrable`.
- Use the static nested class `VehicleStatistics` and the non-static inner class `VehicleInsurance` as specified above.
- All sorting is implemented with Lambda Expressions (no standalone Comparator classes); searching and filtering are implemented with the Stream API; method references replace lambdas where appropriate.
- Search operations return `Optional<Vehicle>` instead of `null`.
- Registration Date is a `LocalDate`, entered/validated as yyyy-MM-dd.
- All existing functionality from earlier stages of the assignment must remain unchanged and continue to work correctly as new features are layered on.